

FEM2D 二维有限元求解器

架构文档

2026 年 7 月 28 日

目录

1	项目概览	3
2	项目文件树	3
3	核心架构：单元抽象层	4
3.1	设计原则	4
3.2	ElementKernel 协议	5
3.3	CST 与 Q4 对比	5
3.4	单元理论公式	5
3.4.1	CST 三角形单元	5
3.4.2	Q4 四边形单元	6
3.4.3	本构矩阵（CST 与 Q4 共用）	7
3.4.4	von Mises 等效应力	7
4	数据层	8
4.1	Mesh 数据类	8
4.2	网格导入	8
4.3	网格拓扑构建（build_connectivity）	9
4.4	约束检查（check_rigid_body_constraints）	9
5	求解管线	10
5.1	预处理阶段（组装前校验）	10
5.2	组装阶段	10
5.3	约束施加与求解	11

目录	2
6 后处理	11
6.1 应力计算与恢复	11
6.2 Z2 误差估计	12
6.3 可视化	13
7 边界检测与分类	13
7.1 自动几何检测 (detect 函数)	13
7.2 Physical Curve 优先路径	14
7.3 边名称解析	14
7.4 与 Q4 的兼容性	14
8 Gmsh 集成	15
9 测试	15
10 模块依赖图	16
11 代码设计模式	17
11.1 惰性求值 (Lazy Evaluation)	17
11.2 策略模式 (Strategy Pattern)	17
11.3 注册表模式 (Registry Pattern)	17
11.4 单一职责分离	17
11.5 防御性编程	18
11.6 向量化优先	18
12 加新单元类型的步骤	19

项目概览

FEM2D 是一个二维线弹性有限元求解器，参照 Bathe 《Finite Element Procedures》（第 2 版）实现。支持三节点常应变三角形（CST）和四节点双线性四边形（Q4）两种单元类型。

- 语言：Python 3.9+，依赖 numpy / scipy / matplotlib
- 规模：~8,800 行，23 个核心模块，107 项测试
- 输入：Abaqus .inp（CPS3/CPE3/CPS4/CPE4）、Gmsh .geo、.spec 规格文件
- 输出：位移/应力/应变云图、Z2 误差估计、等应力带图

项目文件树

1	FEM2D_project/	
2	-- run.py	CLI 入口（791 行）
3	-- run_demo.py	演示脚本
4	-- requirements.txt	
5	-- ARCHITECTURE.md	架构说明
6		
7	-- fem2d/	核心库（23 模块，~6,400 行）
8	-- __init__.py	公共 API 导出
9	-- mesh.py	网格数据类
10	-- material.py	材料本构（D 矩阵 + von Mises）
11	-- preprocess.py	网格导入（.inp / .spec / .geo 配置）
12	-- element_base.py	单元抽象协议（ElementKernel ABC）
13	-- element_cst.py	CST 三角形（CPS3/CPE3/C2D3）
14	-- element_q4.py	Q4 四边形（CPS4/CPE4）
15	-- element_registry.py	单元注册枢纽
16	-- element.py	导出壳（向后兼容）
17	-- assembly.py	总刚组装（向量化 COO）
18	-- solver.py	求解管线
19	-- boundary.py	位移约束（消去法 / 乘大数法）
20	-- loads.py	载荷 API 包装
21	-- loads_core.py	等效节点力（体力/面力/压力）
22	-- stress.py	应力计算 + 4 种应力恢复
23	-- spr.py	SPR patch recovery
24	-- error_est.py	Z2 能量误差 + 牵引跳跃 + 显式残差
25	-- visualize.py	可视化（Gouraud / Isoband / 交互）
26	-- boundary_detect.py	边界检测 + 边名解析
27	-- quality.py	网格质量评分
28	-- patch_test.py	分片检验

```

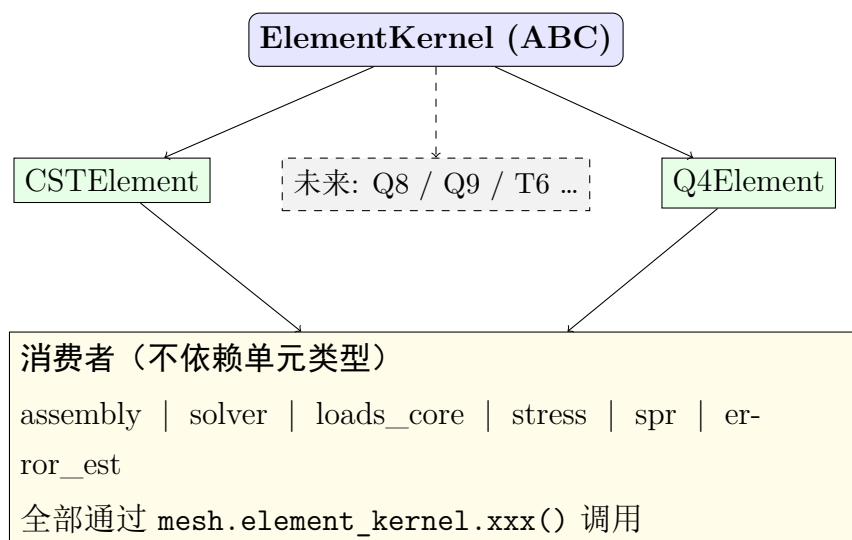
29 | |-- convergence.py      收敛性研究
30 |
31 | |-- scripts/           工具脚本
32 | | |-- geo_spec.py      中文几何描述 → Gmsh .geo
33 | | |-- gmsh_runner.py   Gmsh 安全执行器
34 | | |-- convergence_study.py 批量收敛性研究
35 | | +-- ... (Abaqus 对比脚本)
36 |
37 | |-- tests/             测试 (11文件, 107项)
38 | | |-- test_q4.py       Q4 单元验证
39 | | |-- test_gmsh_quad.py Gmsh 四边形验证
40 | | |-- test_element_abstraction.py 架构抽象验证
41 | | +-- ... (回归 / 应力 / SPR / 极端工况)
42 |
43 | |-- models/            示例网格
44 | | |-- cantilever.inp   悬臂梁 (CPS3)
45 | | |-- plate_hole.inp  带孔板 (CPS3)
46 | | |-- q4_cantilever.inp 悬臂梁 (CPS4)
47 | | |-- q4_plane_strain_cpe4.inp 平面应变 (CPE4)
48 | | |-- plate_q4.geo     四边形生成示例
49 | | +-- ...

```

核心架构：单元抽象层

设计原则

求解器的核心设计是单元类型与求解管线完全解耦。所有单元通过统一的抽象协议接入，加新单元类型只需实现一个 Kernel 类并注册，无需修改任何求解代码。



ElementKernel 协议

每个单元类型必须实现以下方法：

方法	返回	说明
<code>build_geometry(nodes, elements)</code>	dict	面积/质心/系数/Jacobian
<code>stiffness_batch(mesh)</code>	(ne, ndof, ndof)	批量单元刚度
<code>compute_response(mesh, u_e)</code>	(stress, strain, vm)	应力/应变/Mises
<code>jacobian_determinants(mesh)</code>	(ne, nsample)	积分点 det(J)
<code>body_force_vector(mesh, eid, bf)</code>	(ndof,)	体力等效载荷
<code>shape_values_at(coords, x, y)</code>	(nnode,) or None	点定位
<code>verify_mesh(mesh)</code>	bool	完备性 + 刚体模态
<code>recovery_quadrature(mesh, eid)</code>	(N, dA)	Z2/SPR 积分点 (可选)
<code>response_at_quadrature(mesh, u_e)</code>	(stress, strain, dA)	积分点响应 (可选)

CST 与 Q4 对比

属性	CSTElement	Q4Element
注册名	CST / CPS3 / CPE3 / C2D3	Q4 / CPS4 / CPE4
节点数	3	4
每单元 DOF	6	8
边数	3	4
积分方案	1 点（精确）	2×2 Gauss
应力分布	单元内常数	单元内双线性变化
恢复积分	3 点 Hammer	2×2 Gauss
形函数	面积坐标 L_i	$N_i = \frac{1}{4}(1 \pm \xi)(1 \pm \eta)$

单元理论公式

3.4.1 CST 三角形单元

面积坐标。三角形内任意点 $P(x, y)$ 的面积坐标为：

$$L_1 = \frac{A_1}{A}, \quad L_2 = \frac{A_2}{A}, \quad L_3 = \frac{A_3}{A}, \quad L_1 + L_2 + L_3 = 1 \quad (1)$$

形函数。CST 的形函数即为面积坐标本身（Bathe §5.3.2 Eq 5.44）：

$$N_i(x, y) = L_i = \frac{a_i + b_i x + c_i y}{2A} \quad (i = 1, 2, 3) \quad (2)$$

其中系数由节点坐标循环置换得到（Bathe Eq 5.43）：

$$\begin{aligned}
a_1 &= x_2 y_3 - x_3 y_2, & b_1 &= y_2 - y_3, & c_1 &= x_3 - x_2 \\
a_2 &= x_3 y_1 - x_1 y_3, & b_2 &= y_3 - y_1, & c_2 &= x_1 - x_3 \\
a_3 &= x_1 y_2 - x_2 y_1, & b_3 &= y_1 - y_2, & c_3 &= x_2 - x_1
\end{aligned} \tag{3}$$

B 矩阵。应变-位移关系 $\varepsilon = \mathbf{B} \cdot \mathbf{u}^e$ (Bathe Eq 5.43):

$$\mathbf{B} = \frac{1}{2A} \begin{bmatrix} b_1 & 0 & b_2 & 0 & b_3 & 0 \\ 0 & c_1 & 0 & c_2 & 0 & c_3 \\ c_1 & b_1 & c_2 & b_2 & c_3 & b_3 \end{bmatrix}_{3 \times 6} \tag{4}$$

CST 的特殊性: b_i, c_i 为常数, $\mathbf{B}$ 在单元内处处相同 $\Rightarrow$ 应变/应力在单元内为常数。
单元刚度。1 点积分精确 (Bathe Eq 5.27):

$$\mathbf{K}^e = t \cdot A \cdot \mathbf{B}^T \mathbf{D} \mathbf{B} \tag{5}$$

恢复积分。3 点 Hammer 积分 (Bathe Table 5.8):

$$\begin{aligned}
\text{积分点: } & \left(\frac{2}{3}, \frac{1}{6}, \frac{1}{6}\right), \left(\frac{1}{6}, \frac{2}{3}, \frac{1}{6}\right), \left(\frac{1}{6}, \frac{1}{6}, \frac{2}{3}\right) \\
\text{权重: } & w = \frac{1}{3}, \quad \int_{\Omega} f \, dA = A \sum_{i=1}^3 w_i f_i
\end{aligned} \tag{6}$$

3.4.2 Q4 四边形单元

自然坐标。Q4 在参考正方形 $[-1, 1] \times [-1, 1]$ 上定义, 物理坐标通过等参映射:

$$x(\xi, \eta) = \sum_{i=1}^4 N_i(\xi, \eta) x_i, \quad y(\xi, \eta) = \sum_{i=1}^4 N_i(\xi, \eta) y_i \tag{7}$$

形函数 (双线性)。四个节点的形函数 (节点顺序: 左下 $\rightarrow$ 右下 $\rightarrow$ 右上 $\rightarrow$ 左上):

$$\begin{aligned}
N_1 &= \frac{1}{4}(1 - \xi)(1 - \eta) \quad (\text{左下}) \\
N_2 &= \frac{1}{4}(1 + \xi)(1 - \eta) \quad (\text{右下}) \\
N_3 &= \frac{1}{4}(1 + \xi)(1 + \eta) \quad (\text{右上}) \\
N_4 &= \frac{1}{4}(1 - \xi)(1 + \eta) \quad (\text{左上})
\end{aligned} \tag{8}$$

$$N_i(\xi, \eta) = \frac{1}{4}(1 + \xi_i \xi)(1 + \eta_i \eta)$$

“双线性”的含义: 将 N_1 展开为 $\frac{1}{4}(1 - \xi - \eta + \xi\eta)$ ——含常数项、 ξ 项、 η 项、以及双线性交叉项 $\xi\eta$ 。这使得:

- 沿每条边 ($\xi = \pm 1$ 或 $\eta = \pm 1$), $\xi\eta$ 退化为 $\pm\eta$ 或 $\pm\xi \rightarrow$ 边保持直线
- 单元内部 $\xi\eta$ 项使应变可以线性变化 (CST 只能常应变)
- 完备性: $\sum N_i = 1$ 恒成立; 刚体模态: $\mathbf{B} \cdot \mathbf{u}_{rb} = \mathbf{0}$

Jacobi 矩阵。自然坐标到物理坐标的变换：

$$\mathbf{J} = \begin{bmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial y}{\partial \xi} \\ \frac{\partial x}{\partial \eta} & \frac{\partial y}{\partial \eta} \end{bmatrix} = \begin{bmatrix} \sum \frac{\partial N_i}{\partial \xi} x_i & \sum \frac{\partial N_i}{\partial \xi} y_i \\ \sum \frac{\partial N_i}{\partial \eta} x_i & \sum \frac{\partial N_i}{\partial \eta} y_i \end{bmatrix}, \quad \frac{\partial N_i}{\partial \xi} = \frac{\xi_i}{4}(1 + \eta_i \eta), \quad \frac{\partial N_i}{\partial \eta} = \frac{\eta_i}{4}(1 + \xi_i \xi) \quad (9)$$

B 矩阵。物理坐标下的应变-位移矩阵通过 $\mathbf{J}^{-1}$ 转换：

$$\begin{bmatrix} \frac{\partial N_i}{\partial x} \\ \frac{\partial N_i}{\partial y} \end{bmatrix} = \mathbf{J}^{-1} \begin{bmatrix} \frac{\partial N_i}{\partial \xi} \\ \frac{\partial N_i}{\partial \eta} \end{bmatrix}, \quad \mathbf{B}_i = \begin{bmatrix} \frac{\partial N_i}{\partial x} & 0 \\ 0 & \frac{\partial N_i}{\partial y} \\ \frac{\partial N_i}{\partial y} & \frac{\partial N_i}{\partial x} \end{bmatrix}_{3 \times 2}, \quad \mathbf{B} = [\mathbf{B}_1 \ \mathbf{B}_2 \ \mathbf{B}_3 \ \mathbf{B}_4]_{3 \times 8} \quad (10)$$

单元刚度 (2×2 Gauss 全积分)。4 个积分点，权重均为 1：

$$\mathbf{K}^e = t \sum_{q=1}^4 \mathbf{B}^T(\xi_q, \eta_q) \mathbf{D} \mathbf{B}(\xi_q, \eta_q) \det \mathbf{J}(\xi_q, \eta_q) \quad (11)$$

$$\text{Gauss 点: } (\pm \frac{1}{\sqrt{3}}, \pm \frac{1}{\sqrt{3}}), \quad \det \mathbf{J} > 0 \text{ 为物理单元的面积元} \quad (12)$$

3.4.3 本构矩阵 (CST 与 Q4 共用)

平面应力 ($\sigma_{zz} = 0$, Bathe Table 4.3):

$$\mathbf{D}_{\text{stress}} = \frac{E}{1 - \nu^2} \begin{bmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & \frac{1-\nu}{2} \end{bmatrix} \quad (13)$$

平面应变 ($\varepsilon_{zz} = 0$):

$$\mathbf{D}_{\text{strain}} = \frac{E}{(1 + \nu)(1 - 2\nu)} \begin{bmatrix} 1 - \nu & \nu & 0 \\ \nu & 1 - \nu & 0 \\ 0 & 0 & \frac{1-2\nu}{2} \end{bmatrix} \quad (14)$$

3.4.4 von Mises 等效应力

$$\sigma_{\text{vm}} = \begin{cases} \sqrt{\sigma_x^2 + \sigma_y^2 - \sigma_x \sigma_y + 3\tau_{xy}^2} & \text{平面应力} \\ \sqrt{\frac{1}{2}[(\sigma_x - \sigma_y)^2 + (\sigma_y - \sigma_z)^2 + (\sigma_z - \sigma_x)^2] + 3\tau_{xy}^2} & \text{平面应变} \end{cases} \quad (15)$$

平面应变时 $\sigma_z = \nu(\sigma_x + \sigma_y)$ 。

数据层

Mesh 数据类

`mesh.py` (613 行) 是项目中唯一不受单元类型影响的中央数据结构。

```
@dataclass
class Mesh:
    nodes: np.ndarray          # (n_nodes, 2)
    elements: np.ndarray       # (n_elem, nnode_e) ← 列数由 kernel 决定
    elem_type: str = "CPS3"    # 自动解析对应 kernel
    thickness: float = 1.0
    E: float = 210e9
    nu: float = 0.3
    plane_type: str = "stress"

    # 约束与载荷
    fixed_dofs: np.ndarray
    prescribed_vals: dict
    body_force: object
    surface_tractions: list
    concentrated_forces: list

    # 拓扑 (build_connectivity 惰性计算)
    element_kernel: ElementKernel # ← 核心: 通过它访问一切单元操作
    node_to_elems: list
    elem_neighbors: list
    boundary_edges: list
    edge_to_elems: dict
    areas: np.ndarray
    centroids: np.ndarray
    element_dofs: np.ndarray      # (ne, ndofe) 自动推导
```

`build_connectivity()` 通过 `element_kernel.local_edges` 构建边拓扑,通过 `element_kernel.l` 获取几何缓存。CST 返回 3 点的 `b_coeffs` / `c_coeffs`, Q4 返回 2×2 Gauss 的 B 矩阵和 `detJ`, 互不干扰。

网格导入

`preprocess.py` (477 行) 支持读取 Abaqus `.inp` 格式:


```

_VALID_ELEMENTS = {
    "CPS3": 3, "CPE3": 3, "C2D3": 3,    # 三角形
    "CPS4": 4, "CPE4": 4,                # 四边形
}

```

根据 TYPE= 关键字动态决定读取几个节点索引。自动识别平面应力（CPS）与平面应变（CPE）。混合拓扑（三角 + 四边混在一个网格）会直接拒绝。

网格拓扑构建 (build_connectivity)

mesh.py 的 build_connectivity() 方法是整个求解器的拓扑中枢。它在首次访问时惰性执行（幂等），构建以下数据结构：

1. **node_to_elems** — $O(n_{\text{elem}})$ 扫描，每个节点关联的单元列表
2. **edge_to_elems** — 遍历每个单元的 `element_kernel.local_edges`，使用 min / max 键值无向存储。CST 贡献 3 条边/单元，Q4 贡献 4 条边/单元
3. **elem_neighbors** — 共享边的单元对（恰好 2 个单元共享一条边）
4. **boundary_edges** — 仅属于 1 个单元的边，构成求解域的外边界和内孔
5. **几何缓存** — 调用 `kernel.build_geometry()`，通过 `setattr` 注入面积、质心、形函数系数（CST）或 Gauss 点 Jacobian（Q4）
6. **element_dofs** — (n_elem, n_dof_per_elem) 全局 DOF 索引表

边拓扑的关键设计：通过 `kernel.local_edges` 获取局部边定义，而非硬编码 $(i, j), (j, k), (k, i)$ 。这使边界检测、邻居查找、内部边跳跃计算对所有单元类型透明。

非流形边检测：如果一条边被 ≥ 3 个单元共享，直接抛出异常——二维网格中每条边只能属于 1 或 2 个单元。

约束检查 (check_rigid_body_constraints)

在组装总刚之前，逐连通分量验证是否消除了全部 3 个刚体模态（ x -平动、 y -平动、面内转动）：

1. 用 `scipy.sparse.csgraph.connected_components` 分解连通分量
2. 对每个分量，构造约束矩阵 **R**（每行对应一个被约束的 DOF）：

$$\mathbf{R}_i = \begin{bmatrix} 1 & 0 & -\tilde{y}_i \\ 0 & 1 & \tilde{x}_i \end{bmatrix}, \quad \tilde{x}_i = \frac{x_i - x_0}{h}, \quad \tilde{y}_i = \frac{y_i - y_0}{h}$$

其中 (x_0, y_0) 是分量中心， h 是特征尺寸——中心化 + 归一化保证 `matrix_rank` 对大坐标量级不敏感

3. $\text{rank}(\mathbf{R}) < 3 \Rightarrow$ 存在未约束刚体模态，直接拒绝求解

求解管线

求解流程由 `solver.py` 的 `solve(mesh)` 函数编排。每一步都是可独立替换的模块。

预处理阶段（组装前校验）

1. 拓扑构建—`mesh.build_connectivity()`（幂等，已构建则直接返回）
2. **Jacobian 检查**—`kernel.jacobian_report()`。CST 检查 $2A > 0$, Q4 检查所有 Gauss 点和角点的 $\det \mathbf{J} > 0$ 。区分反向（CW）和退化（零面积）单元
3. 完备性/刚体模态验证（可选）——`--self-test` 触发。CST 用面积坐标验证 $\sum N_i = 1$ 和 $\mathbf{B} \cdot \mathbf{u}_{rb} = 0$ ；Q4 对每个 Gauss 点验证
4. 刚体约束检查—`mesh.check_rigid_body_constraints()`（见 §4.3）

组装阶段

总刚组装（`assembly.py`）：

```
# 1. 调用 kernel 获取批量单刚
Ke = mesh.element_kernel.stiffness_batch(mesh) # (ne, ndofe, ndofe)

# 2. 向量化 COO 散射（无 Python 逐单元循环）
dofs = mesh.element_dofs # (ne, ndofe)
rows = broadcast_to(dofs[:, :, None], (ne, ndofe, ndofe)).ravel()
cols = broadcast_to(dofs[:, None, :], (ne, ndofe, ndofe)).ravel()
K = coo_matrix((Ke.ravel(), (rows, cols)), shape=(nd, nd)).tocsr()

# 3. 对称性检查（相对偏斜 > 1e-10 则报错）
K = 0.5 * (K + K.T)
```

关键：`mesh.element_dofs` 的形状由 `kernel` 决定——CST 为 `(ne,6)`，Q4 为 `(ne,8)`。
组装逻辑本身完全不知道也不关心是几节点单元。

载荷组装（`loads_core.py`）：

- 集中力—直接写入 F_{2n} 或 F_{2n+1}
- 体力—委托给 `kernel.body_force_vector(mesh, eid, body_force)`，使用 `np.add.at` 散射到全局向量

- 面力/压力—3 点 Gauss-Legendre 线积分（精度 degree 5）。边上的形函数为线性（CST 和 Q4 都是），3 点可精确积分至二次分布。压力通过 `mesh.boundary_outward_normal()` 自动计算外法向（从相邻单元的 CCW 局部边方向确定，与调用者传入的节点顺序无关）

约束施加与求解

消去法（默认，Bathe §4.2.2 Eq 4.42-4.45）：

1. 划分自由/约束 DOF: $\mathbf{u} = [\mathbf{u}_a \mid \mathbf{u}_b]^\top$
2. 修正自由 DOF 右端项: $\mathbf{R}'_a = \mathbf{R}_a - \mathbf{K}_{ab}\mathbf{u}_b$
3. 求解缩聚系统: $\mathbf{K}_{aa}\mathbf{u}_a = \mathbf{R}'_a$ （SuperLU，`splu` 显式分解）
4. 回代约束位移，计算支反力: $\mathbf{R}_b = \mathbf{K}_{ba}\mathbf{u}_a + \mathbf{K}_{bb}\mathbf{u}_b - \mathbf{F}_b$

乘大数法（备选，Bathe §4.2.2 p.190）：

$$K_{ii} += k_{\text{penalty}}, \quad F_i += k_{\text{penalty}} \cdot d_i, \quad k_{\text{penalty}} = \max |K_{ii}| \times 10^8$$

加法式避免刚度量纲平方。代价是条件数额外抬高 $\sim 10^8$ 。

残差检查（Bathe §8.2.6）：

$$\text{残差} = \frac{\|\mathbf{Ku} - \mathbf{F}\|_\infty}{\|\mathbf{K}\|_\infty \|\mathbf{u}\|_\infty + \|\mathbf{F}\|_\infty}$$

使用无穷范数（相容范数），残差 $> 10^{-3}$ 直接报错。

全局平衡检查: $\sum \mathbf{F}_{\text{ext}} + \sum \mathbf{R} \approx \mathbf{0}$, $\sum \mathbf{M}_{\text{ext}} + \sum \mathbf{M}_R \approx \mathbf{0}$ 。残差小只说明线性方程解得准，不等于载荷方向/边界条件正确——力平衡是独立验证。

后处理

应力计算与恢复

单元应力。原始 FEM 应力通过 kernel 计算：

- CST: $\sigma^e = \mathbf{DBu}^e$ （单元内常数）
- Q4: $\sigma^e = \frac{1}{A} \sum_q \sigma_q \cdot \Delta A_q$ （Gauss 点面积加权平均）

应力恢复（磨平）。FEM 应力在单元间不连续，需要通过恢复技术得到连续的节点应力场 σ^* 。项目实现了 4 种方法：

1. 简单平均— $\sigma_n^* = \frac{1}{m} \sum_{e \ni n} \sigma^e$ （等权，最简单但忽略面积差异）

2. 面积加权— $\sigma_n^* = \frac{\sum A_e \sigma^e}{\sum A_e}$ (大单元权重大)
3. L2 投影 (Bathe Ex 4.27) —最小化 $\int_{\Omega} (\sigma^* - \sigma^h)^2 d\Omega$, 求解 $\mathbf{M}\sigma^* = \mathbf{f}$, 其中 $\mathbf{M}$ 为一致质量矩阵, $\mathbf{f}$ 为单元应力投影
4. 边界外推—边界节点度数低时 (面积加权外推可能不准), 沿相邻两节点线性外推

SPR (Zienkiewicz-Zhu Patch Recovery)。逐节点的局部最小二乘拟合:

$$\begin{aligned} &\text{对节点 } n, \text{ 收集周围 patch 单元, 拟合 } \hat{\sigma}(x, y) = \mathbf{P}\mathbf{a} \\ &\mathbf{P} = [1, \tilde{x}, \tilde{y}], \quad \tilde{x} = (x - x_n)/h, \quad \tilde{y} = (y - y_n)/h \\ &\mathbf{a} = (\mathbf{P}^T \mathbf{P})^{-1} \mathbf{P}^T \sigma_{\text{patch}} \Rightarrow \sigma_n^* = \hat{\sigma}(x_n, y_n) = a_0 \end{aligned} \quad (16)$$

如果 patch 单元不足 3 个或拟合矩阵病态 ($\kappa > 10^8$), 自动扩展到邻居的邻居 (最多 3 环), 仍不足则退化为均值。

所有恢复方法统一通过 `kernel.recovery_quadrature()` 获取采样点:

- CST: 3 点 Hammer (面积坐标, 每点权 $A/3$) —对线性 σ^* 的二次型精确积分
- Q4: 2×2 Gauss (自然坐标, 权 $\det \mathbf{J}$) —对双线性 σ^* 的二次型精确积分

关键规则 (来自项目经验): **SPR 必须先恢复 $\sigma_x, \sigma_y, \tau_{xy}$ 三个分量, 再从节点分量计算 Mises/主应力。不能先算单元 Mises $\rightarrow$ SPR $\rightarrow$ 节点 Mises——因为 SPR 是线性算子, Mises 是非线性函数, 二者不可交换。**

Z2 误差估计

`error_est.py` (377 行) 实现基于应力恢复的误差评估:

Z2 能量范数误差 (Zienkiewicz-Zhu 1987):

$$\|e\|^2 = \int_{\Omega} (\sigma^* - \sigma^h)^T \mathbf{D}^{-1} (\sigma^* - \sigma^h) d\Omega, \quad \eta = \frac{\|e\|}{\|U\|} \times 100\% \quad (17)$$

其中 $\mathbf{D}^{-1}$ 为柔度矩阵 (对应力分量区别加权), 积分通过 `kernel.recovery_quadrature()` 在恢复积分点上计算。输出每个单元的误差贡献百分比, 指导自适应加密。

与 Abaqus MISESERI 的区别:

	Abaqus MISESERI	本项目的 Z2
计算量	仅 Mises 标量差 (Pa)	全应力张量能量范数 (%)
柔度加权	无	$\mathbf{D}^{-1}$ 加权
全局指标	无	$\eta\%$
逐单元贡献	无	百分比
牵引跳跃图	无	有

牵引跳跃。沿每条内部边计算 $\|(\sigma^+ - \sigma^-)\mathbf{n}\|$ ，归一化后着色。这是 Bathe §4.3.6 等应力带法的基础——高跳跃区域 = 网格不足区域。

显式残差指示器。Verfürth (1996) 型，不依赖应力恢复：

$$\eta_K^2 = h_K^2 \int_K |\mathbf{f}^B|^2 dA + \frac{1}{2} \sum_{e \in \partial K} h_e \int_e \|[\![\sigma^h \mathbf{n}]\!]\|^2 ds + \sum_{e \in \Gamma_t} h_e \int_e |\bar{\mathbf{t}} - \sigma^h \mathbf{n}|^2 ds \quad (18)$$

三项分别对应：体力残差、内部边跳跃、Neumann 边界残差。CST 中 $\nabla \cdot \sigma^h = \mathbf{0}$ （常应力），故第一项仅含体力。Dirichlet 边（固支）跳过（反力未知，不能当自由边处理）。

可视化

`visualize.py` (665 行)：

- Gouraud 着色云图 ($\sigma_x/\sigma_y/\tau_{xy}/\text{Mises}/\sigma_1/\sigma_2/\tau_{\max}$)
- Isoband 等应力带 (Bathe §4.3.6)
- 牵引跳跃边着色
- 变形图（位移放大）
- 交互式按键切换
- 三角形用 Triangulation，四边形自动拆为 2 个三角显示

边界检测与分类

`boundary_detect.py` (1191 行) 提供从网格拓扑自动识别边界边、分割为几何基元、并支持多种命名方式的功能。

自动几何检测（detect 函数）

1. 收集边界边—从 `mesh.boundary_edges` (build_connectivity 通过 `kernel.local_edges` 构建)
2. 连通分量分解—邻接图 $\rightarrow$ 有序闭环。优先从度 = 1 的端点追踪，避免半链阻塞
3. 外边界/内孔区分—每个环的质心对更大环做 ray-casting。奇数层 = 孔，偶数层 = 外边（支持多个断开零件）
4. 离散曲率计算— $\kappa_i = \theta_i/\ell_i$ ， θ_i 为有符号转向角， ℓ_i 为平均边长

5. 双边滤波去噪—保边平滑: $w_{ij} = \exp(-\frac{d_{ij}^2}{2\sigma_s^2}) \cdot \exp(-\frac{|\Delta\kappa|^2}{2\sigma_r^2})$
6. 角点检测 + 曲率分割—三类分割点: 曲率极大值、曲率过零、曲率阶跃 → 基元分类 (直线/圆弧/椭圆/曲线)
7. 后处理—同轴邻接直边合并、按外边优先 + 节点数降序排序

全过程**不依赖单元类型**——只操作节点坐标和无向边对。CST 的 3 条边和 Q4 的 4 条边在拓扑层面对边界检测完全透明。

Physical Curve 优先路径

当 Gmsh .geo 定义了 Physical Curve 时, `segments_from_physical_curves` 优先使用语义名称分界:

1. 从 `preprocess.read_inp_edgesets` 获取每条边的 ELSET 标签
2. `.geo` → `read_geo_groups` → {语义名称 → 曲线 ID} 映射
3. 沿自动分段遍历, 仅在语义名称变化的边处切分
4. 拓扑修正: 连通闭环面积 + ray-casting 奇偶规则纠正内外标签

边名称解析

4 级优先级匹配:

1. 直接标签—Physical Curve 名称 (如 “底部”“椭圆孔”), 不区分大小写子串匹配
2. 中文别名 → 几何方位—左 → left, 底 → bottom, 孔 → hole。通过质心位置判断
3. 快捷字母—a= 圆弧, l= 直边, o= 外边
4. 数字索引—1-based, 兜底

与 Q4 的兼容性

边界检测只依赖 `mesh.boundary_edges` (无向边对), 通过 `kernel.local_edges` 构建。Q4 的 4 条边在拓扑层与 CST 的 3 条边**完全等价**——曲率计算、滤波、分类均只操作节点坐标。

Gmsh 集成

scripts/gmsh_runner.py (211 行) 负责安全的外部网格生成:

- 源文件只读—通过 `-setnumber` CLI 参数控制, 不修改.geo
- 剥离 Save 指令—防止 Gmsh 输出被脚本内 Save 劫持
- 临时输出 + 验证—写入临时文件, 检查拓扑 (纯三角/纯四边, 无混合), 通过后原子发布
- 四边形模式—`--quad` → `-setnumber Mesh.RecombineAll 1 -setnumber Mesh.Algorithm 8`

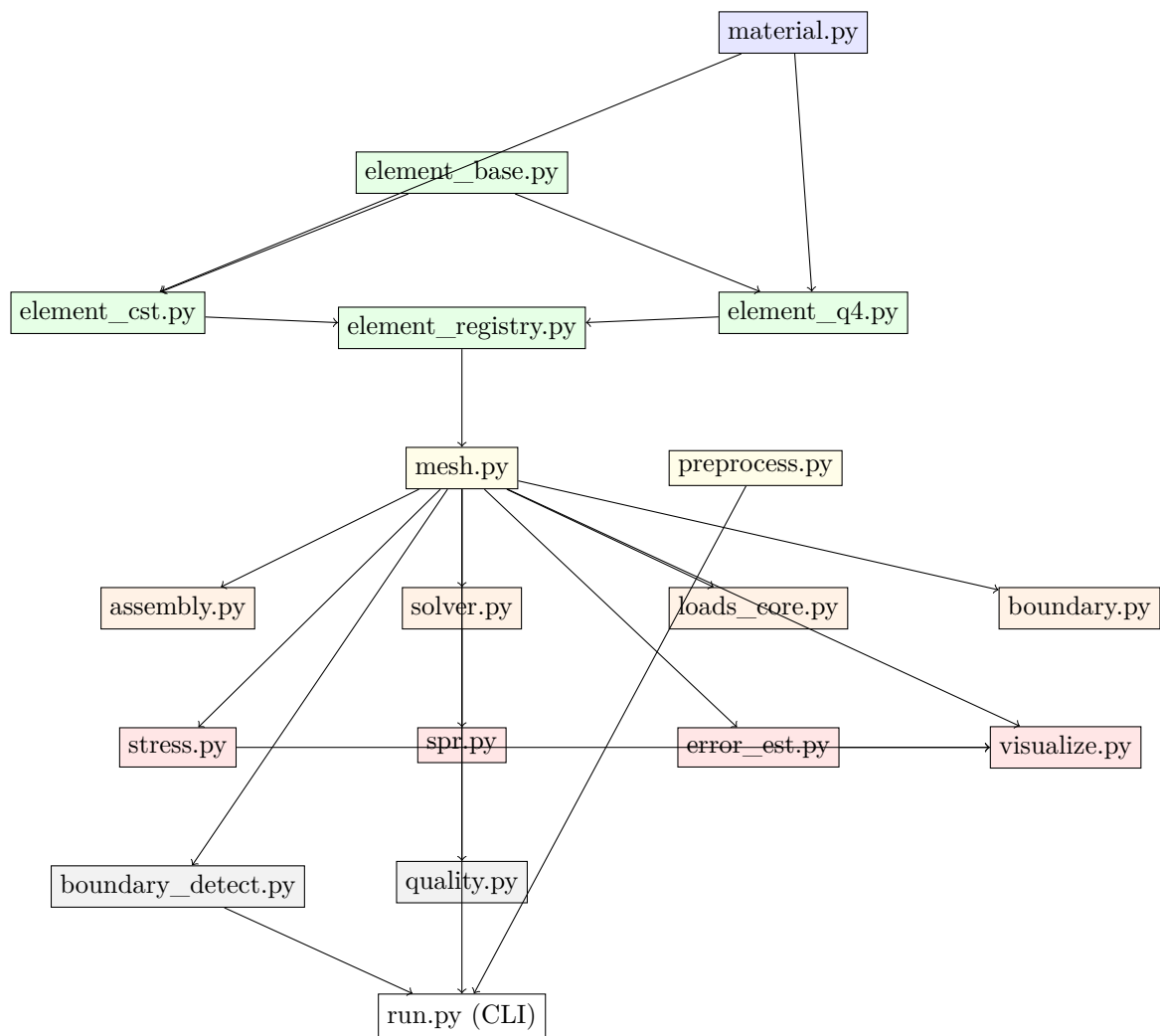
scripts/geo_spec.py 提供中文几何描述语法, 自动生成.geo 并调用 Gmsh。

测试

107 项 pytest 测试, 覆盖:

测试文件	说明
test_q4.py	Q4 单元刚度、响应、Jacobian、体力、验证
test_gmsh_quad.py	Gmsh 四边形命令确定性、拓扑校验
test_element_abstraction.py	4 节点 mock kernel 验证架构通用性
test_regressions.py	回归测试 (边界检测、Physical Curve、CCW 翻转)
test_stress.py	应力恢复、材料参数、载荷组合
test_spr.py	SPR 线性一致性
test_traction_jump.py	牵引跳跃旋转不变性、常应力零跳跃
test_extreme.py	极端网格、退化单元、大变形
test_random.py	随机网格一致性
test_input_guards.py	.inp 解析边界情况
test_isoband.py / test_plot_three.py	可视化
test_patch.py	分片检验

模块依赖图



- 蓝色：材料（所有单元共用）
- 绿色：单元层（协议 + 实现 + 注册）
- 黄色：数据层（网格 + 导入）
- 橙色：求解层（组装 + 求解 + 载荷 + 约束）
- 红色：后处理（应力 + SPR + Z2 + 可视化）
- 灰色：辅助（边界检测 + 质量评分）

代码设计模式

惰性求值 (Lazy Evaluation)

`Mesh.build_connectivity()` 采用惰性求值模式: 拓扑数据 (`node_to_elems`, `elem_neighbors`, `boundary_edges`, `areas`, `centroids`, `element_dofs`) 在首次访问时才计算, 后续调用直接返回。修改 `nodes/elements` 后需显式调用 `invalidate_cache()`。

好处: `Mesh` 构造时只做基本校验 (不跑拓扑), 求解器在需要时才触发计算。避免了“创建 `Mesh` 就要等很久”的问题。

策略模式 (Strategy Pattern)

`ElementKernel` 是策略模式的典型应用: 求解管线 (`assembly`, `solver`, `loads`, `stress`) 是上下文, 具体的单元类型 (`CST`, `Q4`) 是策略。上下文通过 `mesh.element_kernel` 调用策略, 不关心具体实现。

```
# 上下文代码 (assembly.py) —— 不知道也不关心是什么单元
Ke = mesh.element_kernel.stiffness_batch(mesh) # 策略方法
dofs = mesh.element_dofs                      # 策略决定的形状
```

注册表模式 (Registry Pattern)

`element_base.py` 维护全局 `_REGISTRY` 字典。单元类型通过 `register_element(kernel)` 注册自己 (含所有别名)。`get_element_kernel(elem_type)` 按名称查找。重复注册同一键但不同 `kernel` 实例会报错——防止 `import` 顺序静默改变单元行为。

单一职责分离

每个模块只做一件事:

模块	职责（只有一件）
mesh.py	数据容器 + 拓扑构建
material.py	本构关系（D 矩阵 + von Mises）
element_base.py	定义 kernel 协议
element_cst.py / element_q4.py	实现具体单元的数学公式
assembly.py	稀疏矩阵散射
boundary.py	线性代数约束施加
loads_core.py	体力/面力/压力积分为等效节点力
solver.py	流程编排（组装 → 求解 → 校验）
stress.py	应力恢复（磨平）
spr.py	局部最小二乘拟合
error_est.py	误差估计（Z2 + 跳跃 + 残差）
visualize.py	所有 matplotlib 调用
boundary_detect.py	几何边界识别
preprocess.py	文件 I/O 和格式解析

`run.py` 不包含任何 FEM 算法——它只做 CLI 参数解析 + 交互输入 + 流程编排（把上面各模块按顺序串起来）。

防御性编程

输入校验：`Mesh.__post_init__` 校验 NaN/Inf、负索引、越界、重复单元、非流形边、材料参数合法性。校验在组装前快速失败，避免难以调试的奇异矩阵错误。

原子 I/O：`gmsh_runner` 写入临时文件 → 验证拓扑 → `os.replace()` 原子替换。失败时保留旧文件。

异常而非静默：奇异矩阵抛异常（不返回 NaN）、不支持的单元类型抛异常（不跳过）、残差过大抛异常（不让用户拿着错误结果继续分析）。

向量化优先

FEM 的核心操作（刚度组装、应力计算、B 矩阵构造）全部向量化：

- 批量 B 矩阵：`(ne, 3, n_dof_per_elem)` 一次性构造
- 批量单刚：`B.transpose(0,2,1) @ (D @ B)` 无 Python 循环
- COO 散射：`np.broadcast_to` + 一次 `coo_matrix` 调用

LIL 参考实现保留用于教学对比（逐标量写入，慢 $\sim 2.5\times$ ），但生产路径统一使用批量 COO。

加新单元类型的步骤

以加 Q8（八节点四边形）为例：

1. 新建 `element_q8.py`，实现 `ElementKernel` 的 8 个抽象方法
2. 在文件末尾调用 `register_element(Q8Element())`
3. 在 `element_registry.py` 加一行 `from .element_q8 import Q8, Q8Element`
4. 在 `preprocess.py` 的 `_VALID_ELEMENTS` 中加 `"CPS8": 8`
5. `assembly.py` / `solver.py` / `loads_core.py` / `stress.py` / `error_est.py` / `boundary_detect.py`:
无需修改